

TREES

BINARY SEARCH TREE

TREE TRAVERSAL

CSI 219 – Discrete Mathematics

BINARY TREE

ORDERED ROOTED TREE

- **Binary Tree**

An m -ary tree with $m = 2$ is called a *binary tree*.

- **Ordered Root Tree**

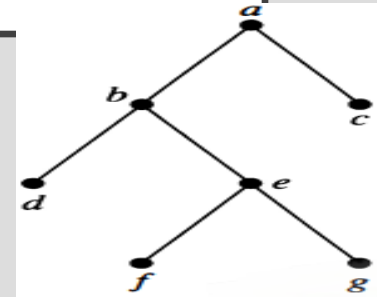
An *ordered rooted tree* is a rooted tree where the children of each internal vertex are ordered.

- **Left and Right Child**

In an ordered binary tree, the first child is called the *left child* and the second child is called the *right child*.

- **Left and Right Subtree**

The tree rooted at the left child is called the *left subtree* and the tree rooted at the right child is called the *right subtree*.

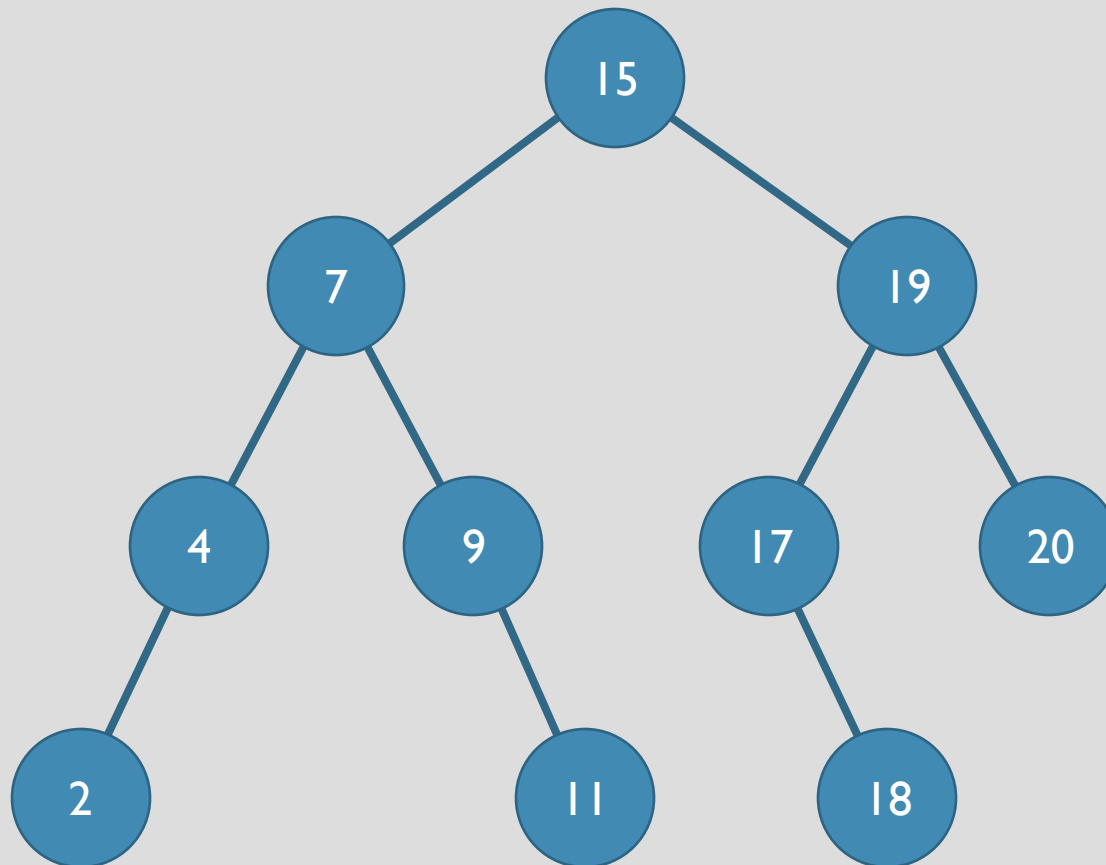


BINARY SEARCH TREE (BST)

- A data structure that is used to search efficiently
- Ordered tree
 - The order of the children of a parent is important
 - Two children – left child and right child

Create a binary search tree using the following data:

15, 7, 19, 9, 4, 2, 17, 11, 20, 18

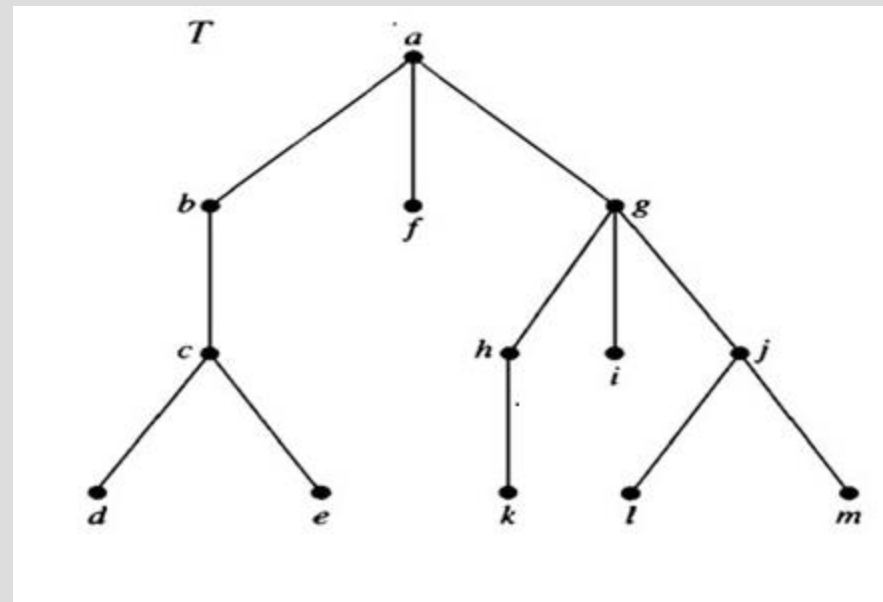


NOTES ABOUT BST

- You might be asked to draw binary search trees for objects other than numbers (like strings)
- For strings, consider dictionary order (e.g. center < centre < dictator)
- For other objects, the order conditions will be given

TREE TRAVERSAL

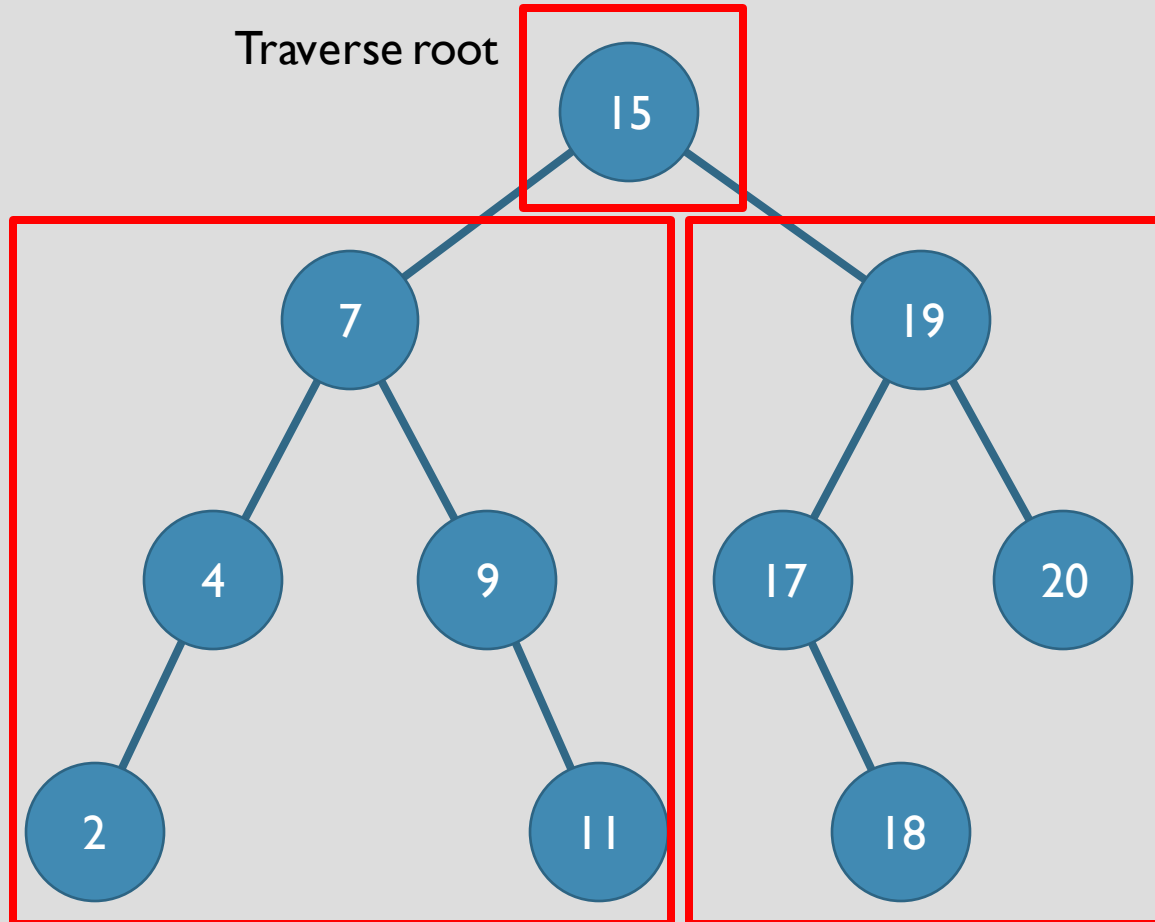
- There are some tree traversal standards
 - Preorder: Root, Left, Right
 - Inorder: Left, Root, Right
 - Postorder: Left, Right, Root
- In each case –
 - Root = Root node
 - Left = Left subtree
 - Right = Right subtree



PREORDER TRAVERSAL: ROOT, LEFT, RIGHT

15,

Traverse root

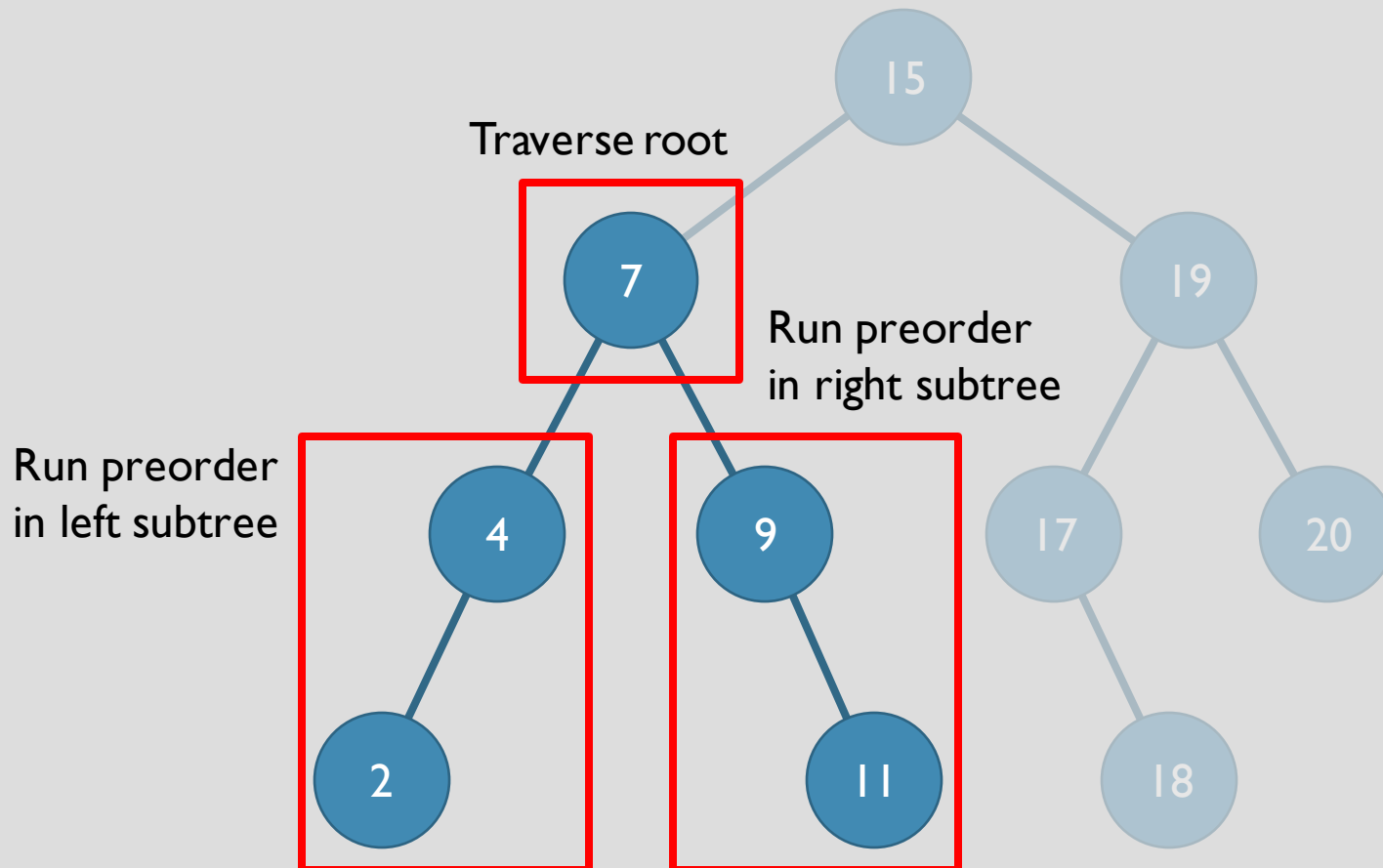


Run preorder
in left subtree

Run preorder
in right subtree

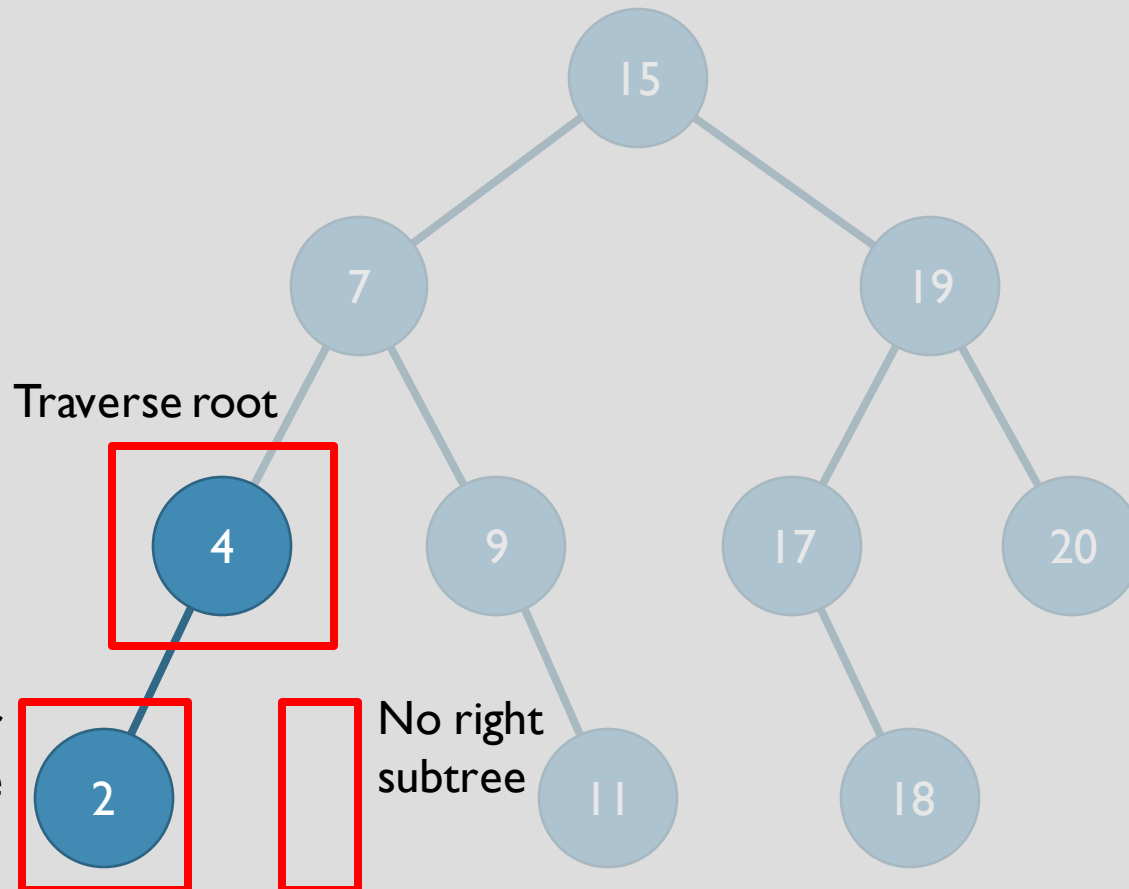
PREORDER TRAVERSAL

15, 7,



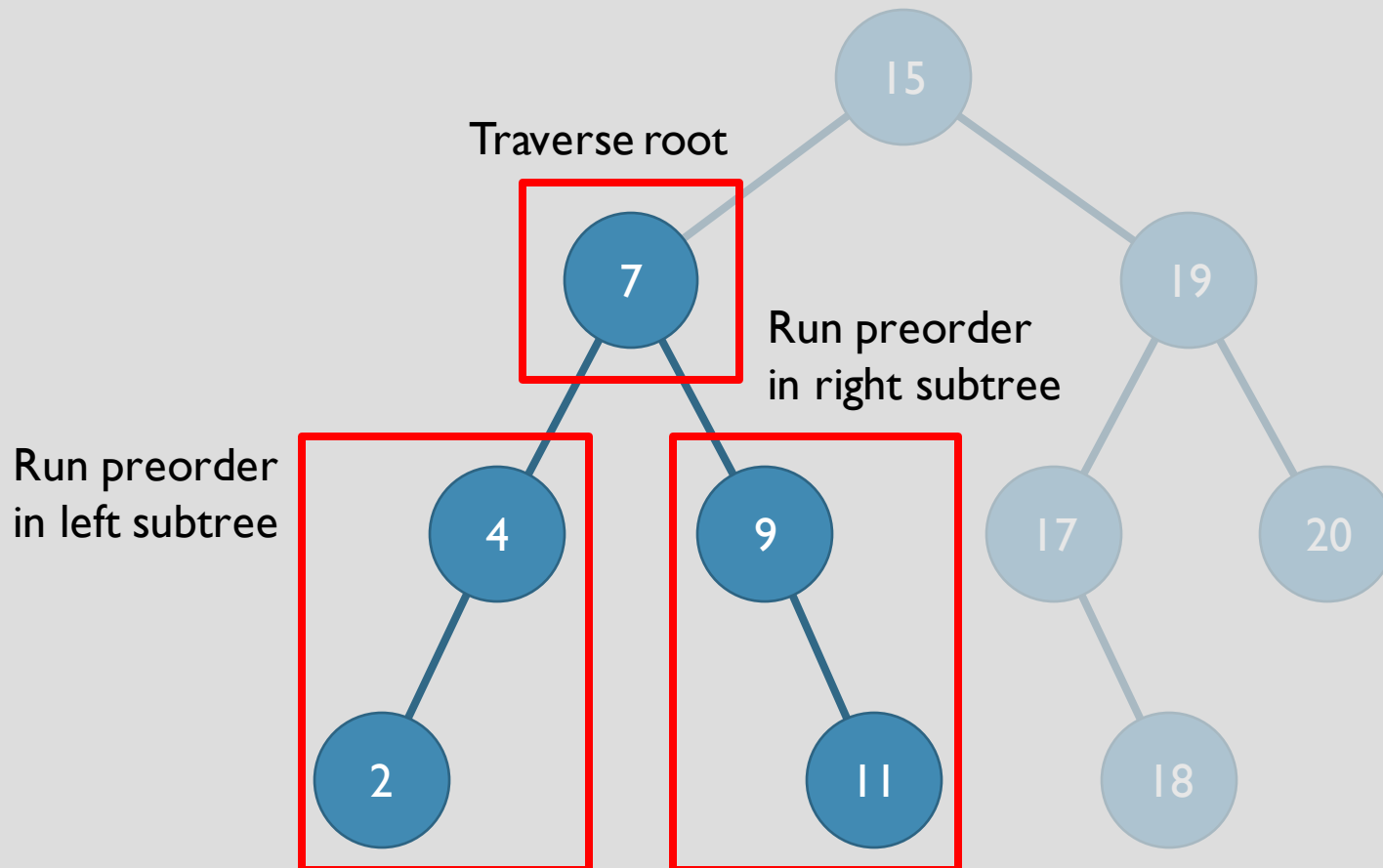
PREORDER TRAVERSAL

15, 7, 4, 2,



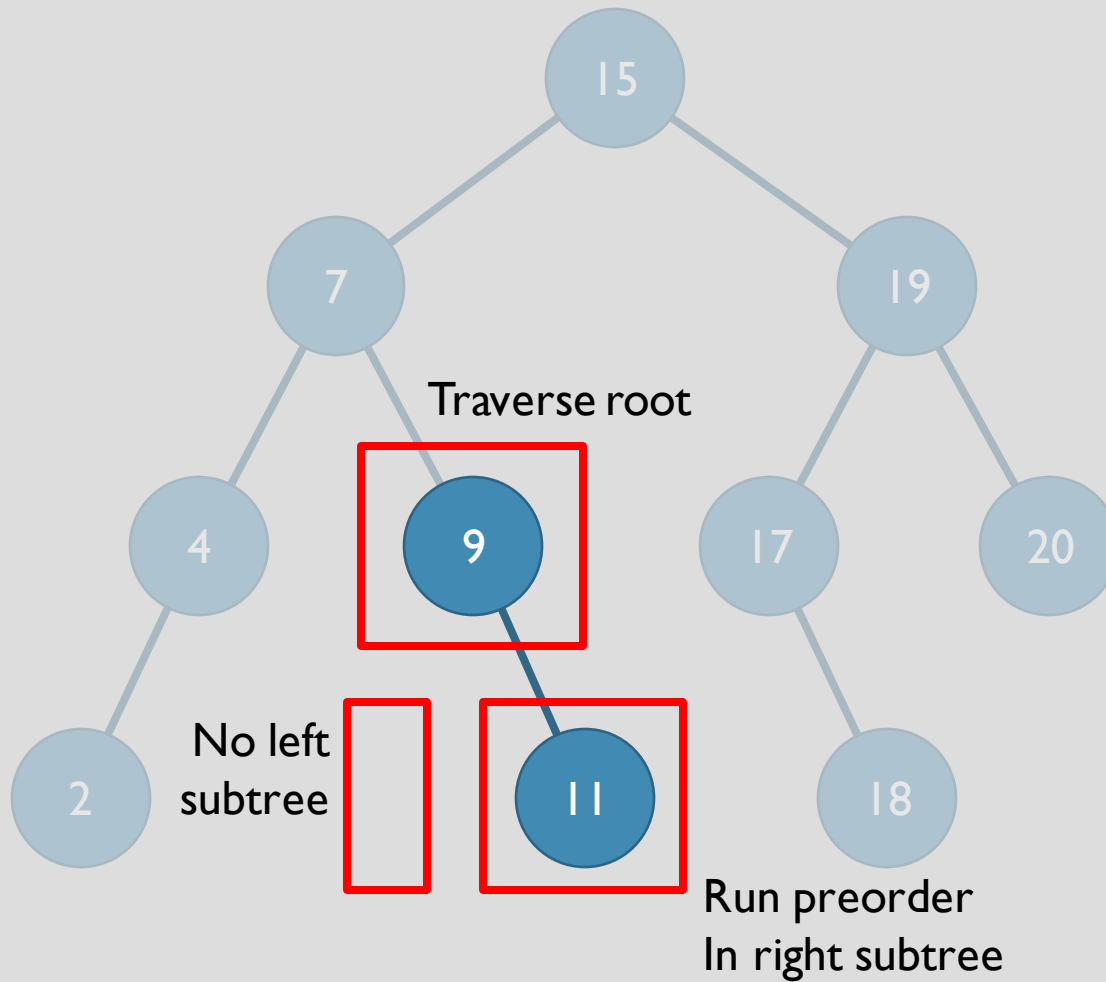
PREORDER TRAVERSAL

15, 7, 4, 2,



PREORDER TRAVERSAL

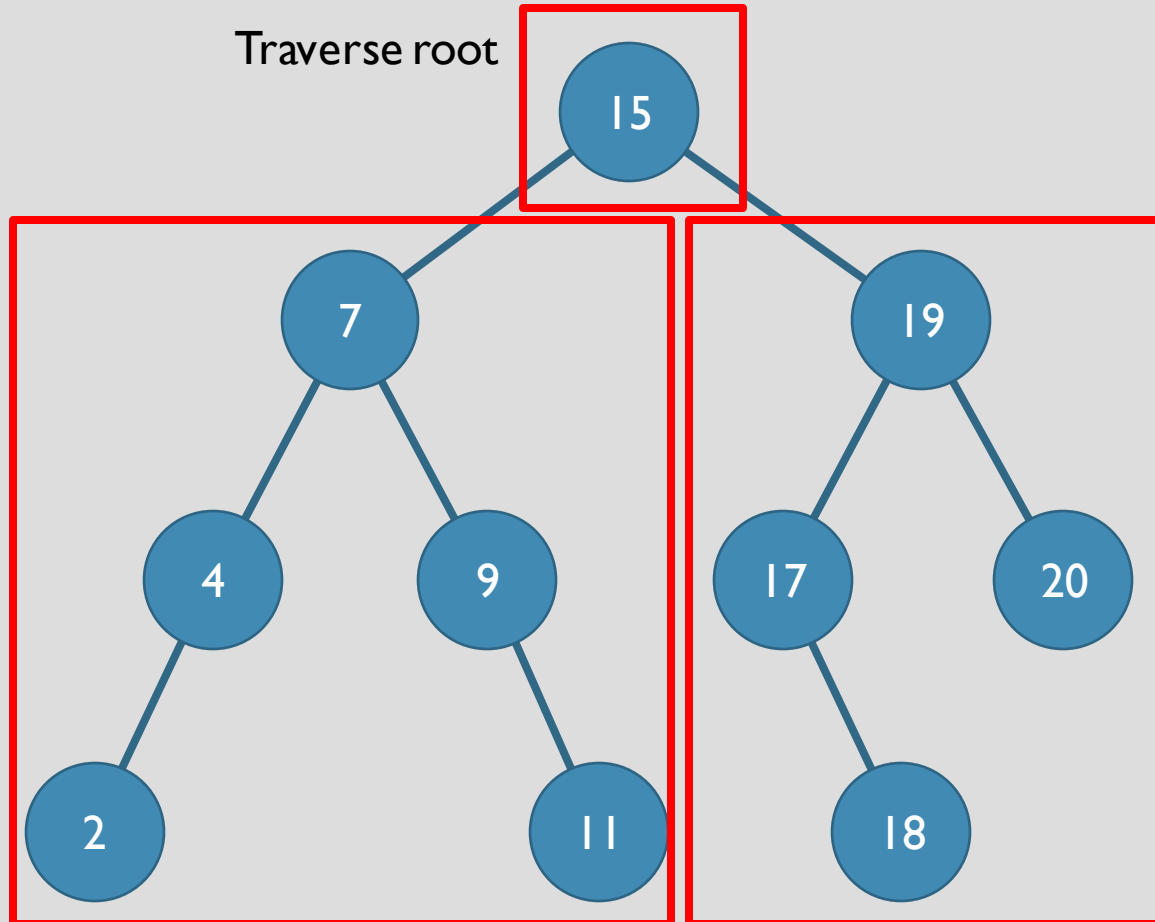
15, 7, 4, 2, 9, 11,



PREORDER TRAVERSAL

15, 7, 4, 2, 9, 11,

Traverse root

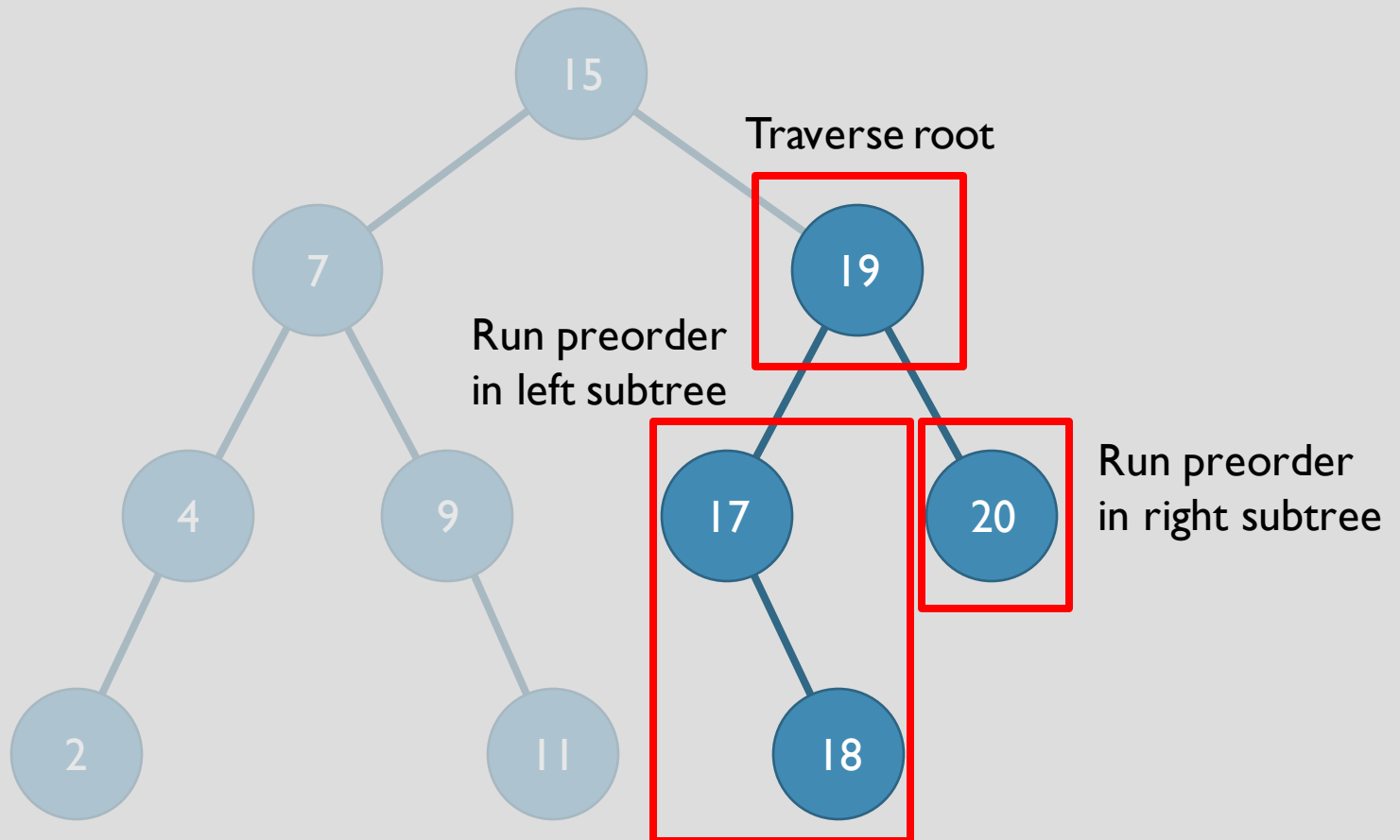


Run preorder
in left subtree

Run preorder
in right subtree

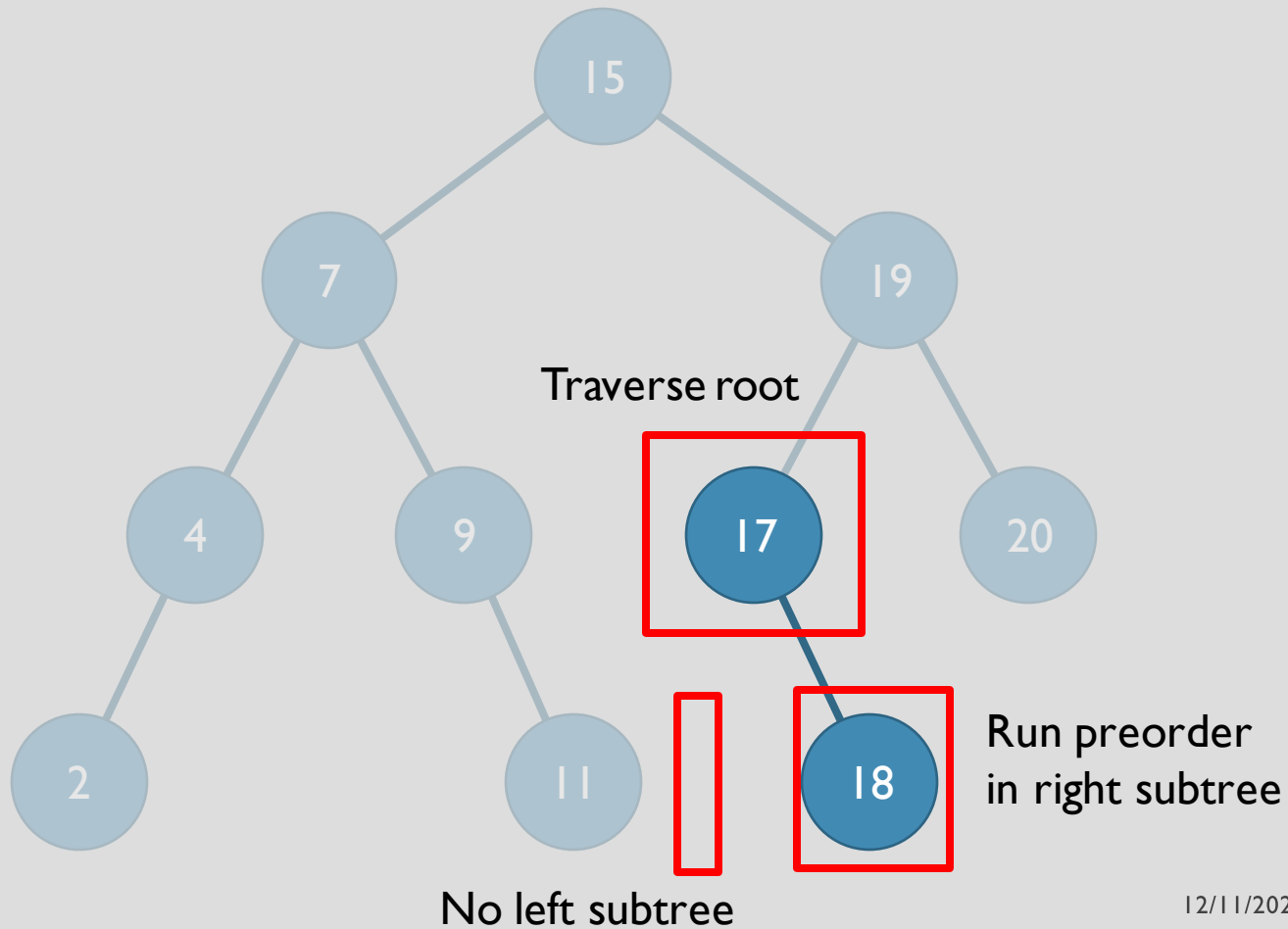
PREORDER TRAVERSAL

15, 7, 4, 2, 9, 11, 19,



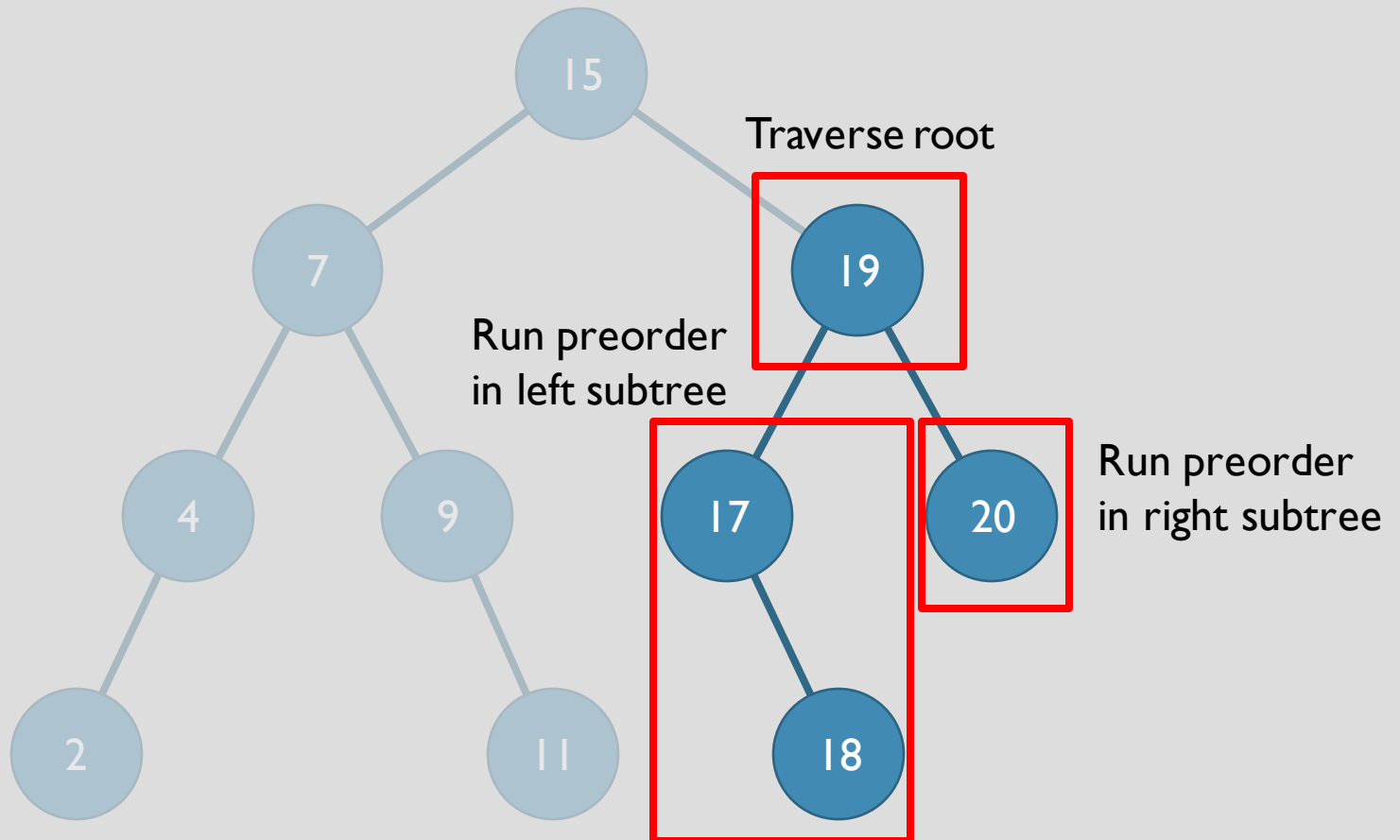
PREORDER TRAVERSAL

15, 7, 4, 2, 9, 11, 19, 17, 18,

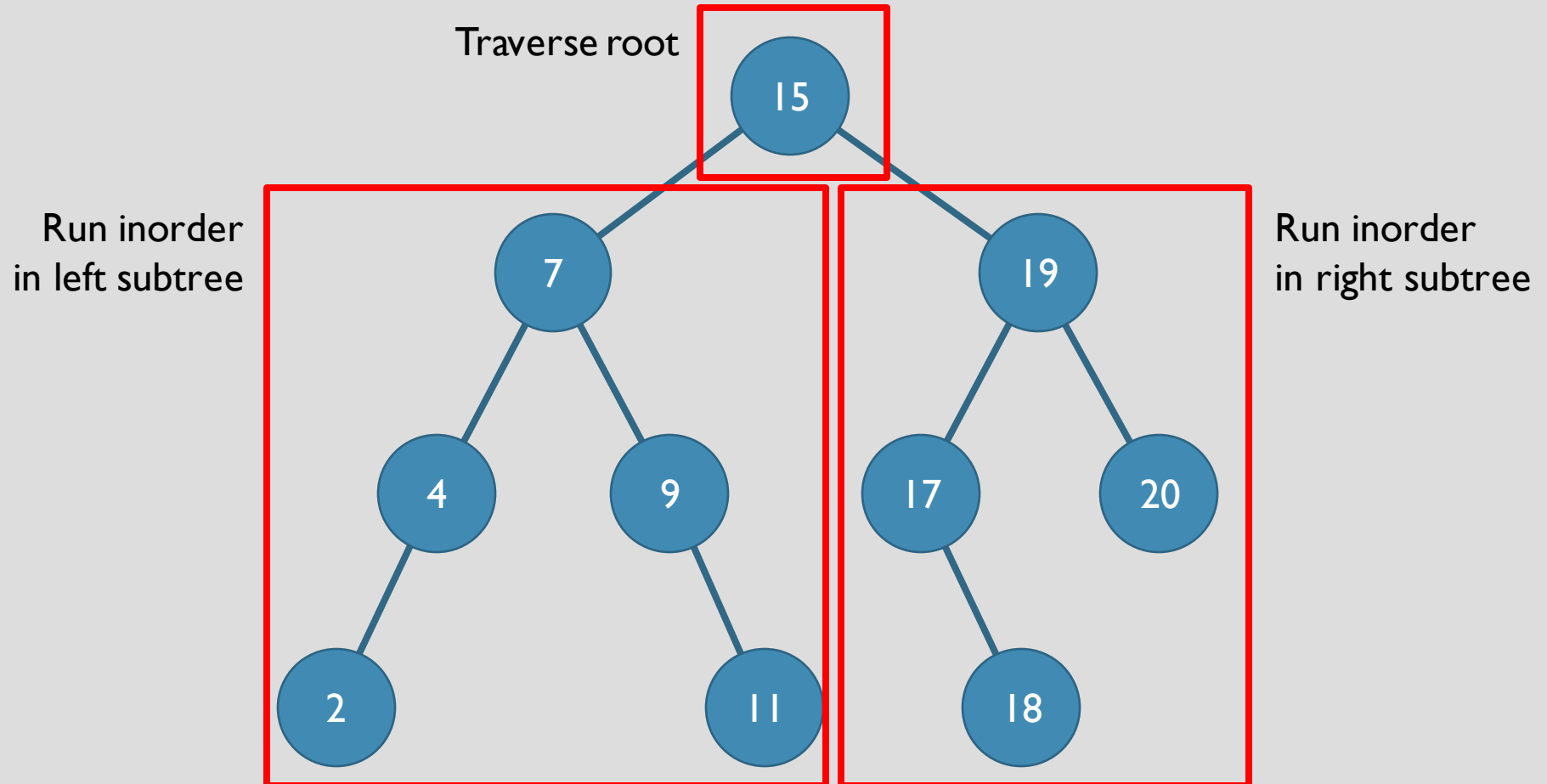


PREORDER TRAVERSAL

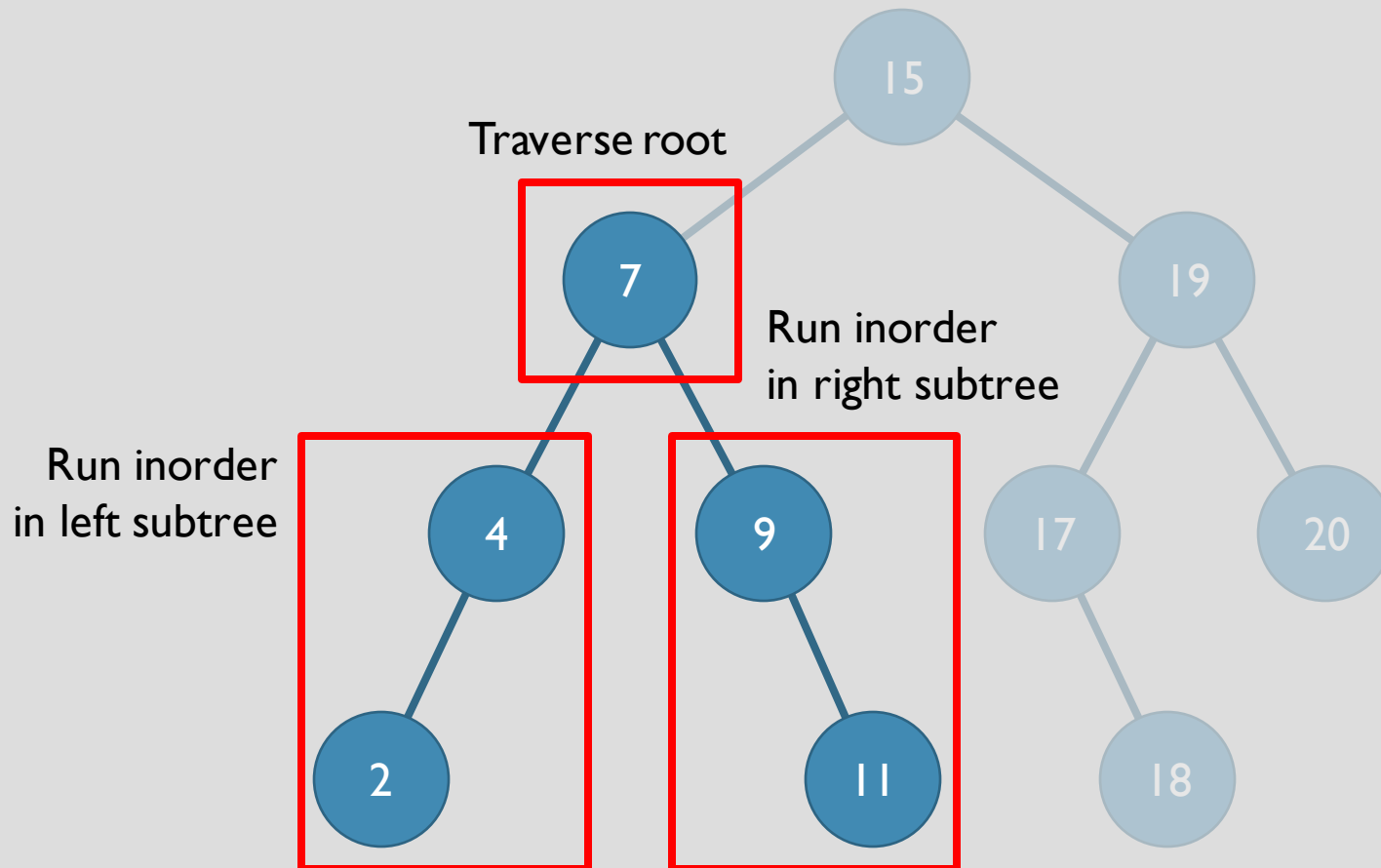
15, 7, 4, 2, 9, 11, 19, 17, 18, 20



INORDER TRAVERSAL: LEFT, ROOT, RIGHT

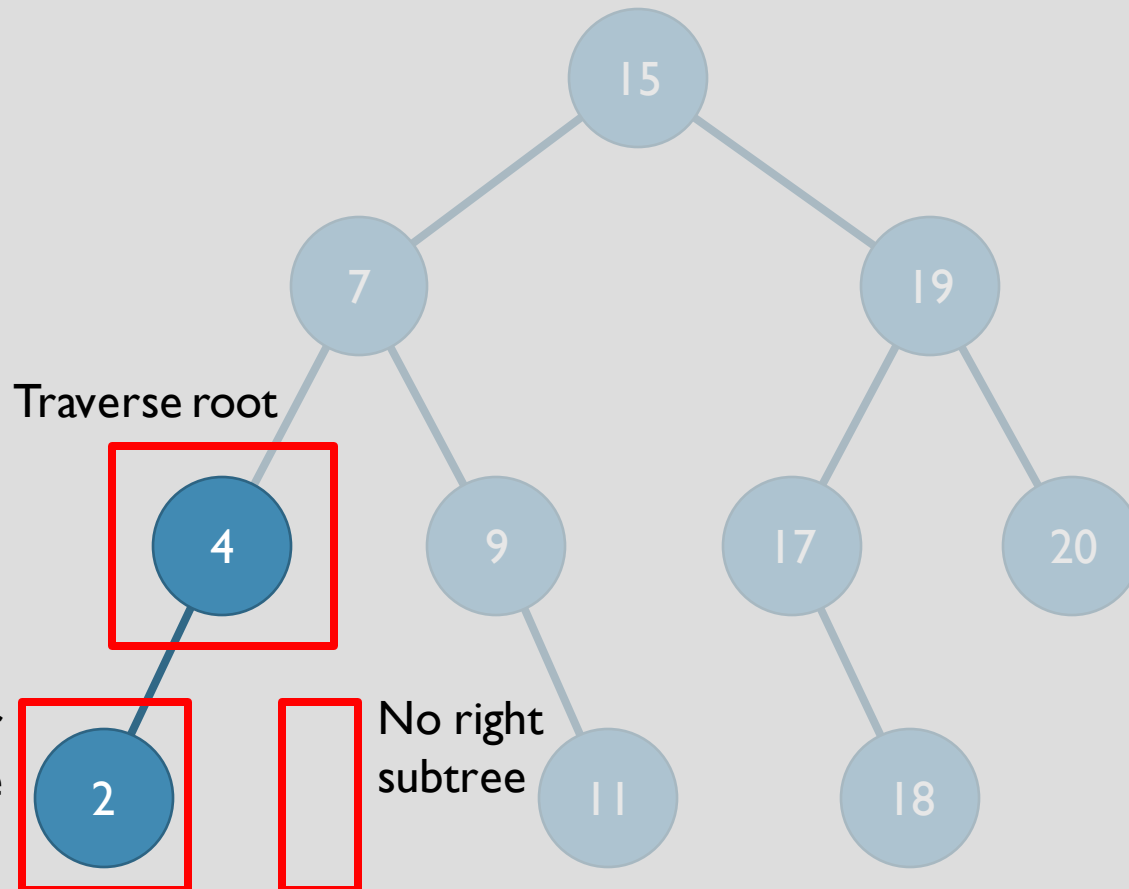


INORDER TRAVERSAL



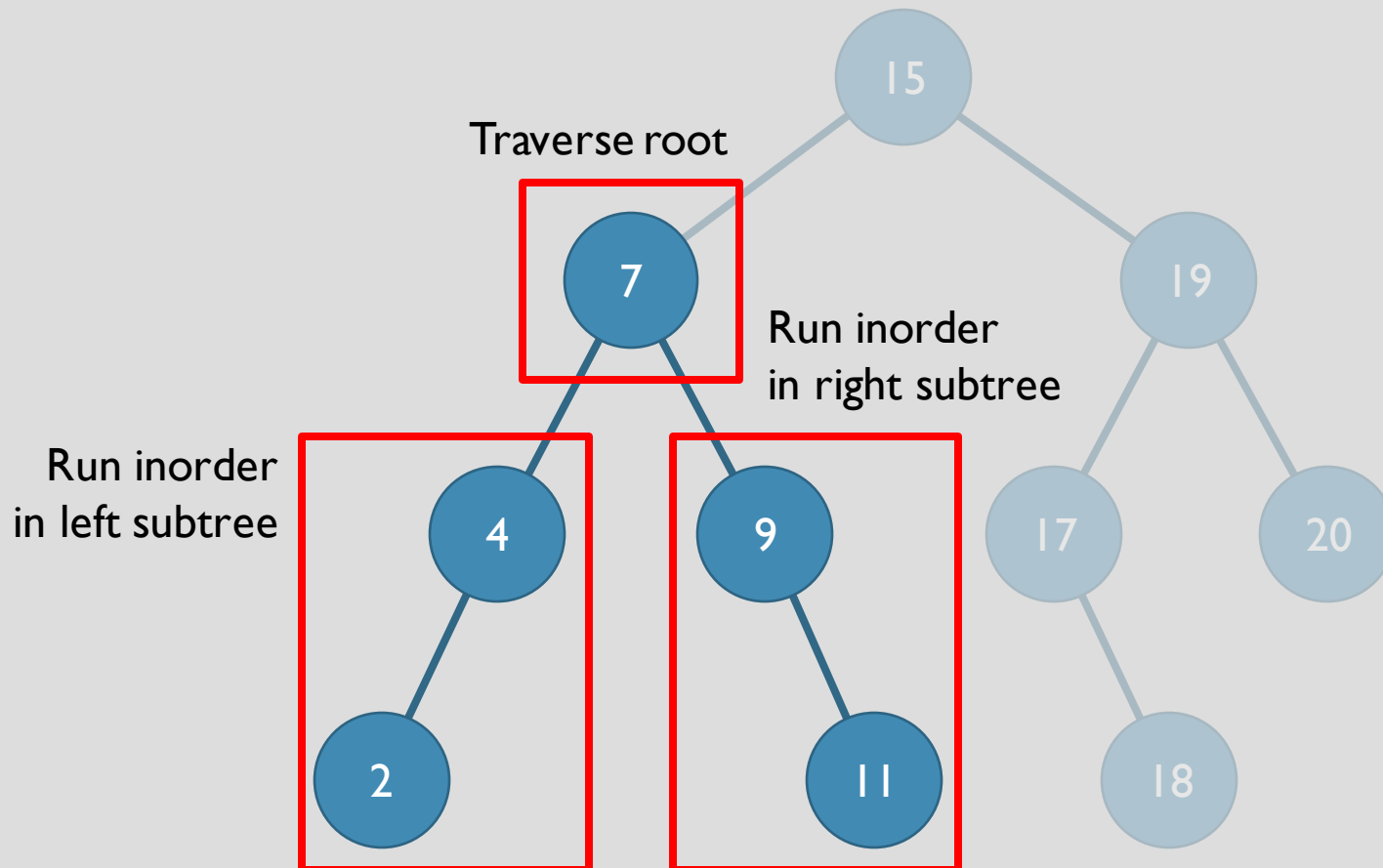
INORDER TRAVERSAL

2, 4,



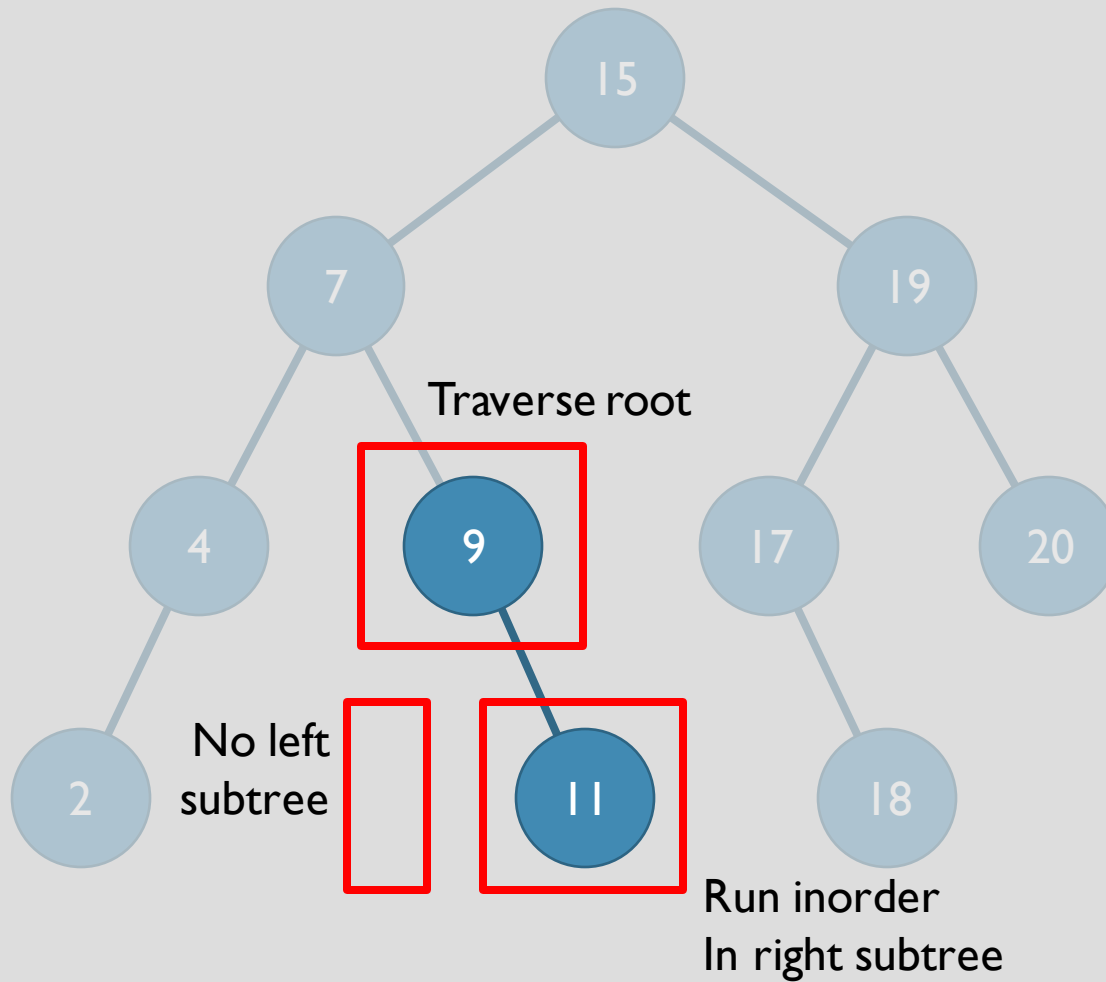
INORDER TRAVERSAL

2, 4, 7,



INORDER TRAVERSAL

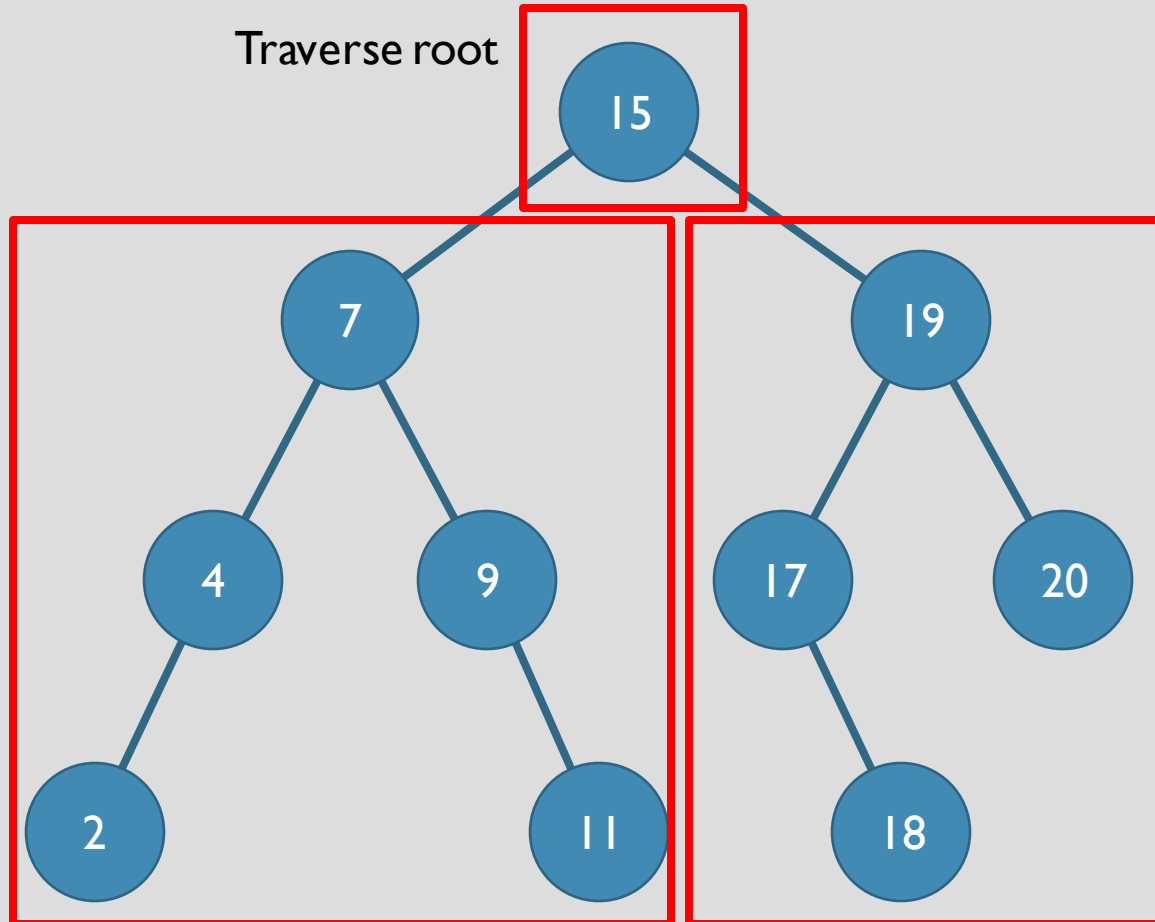
2, 4, 7, 9, 11,



INORDER TRAVERSAL

2, 4, 7, 9, 11, 15,

Traverse root

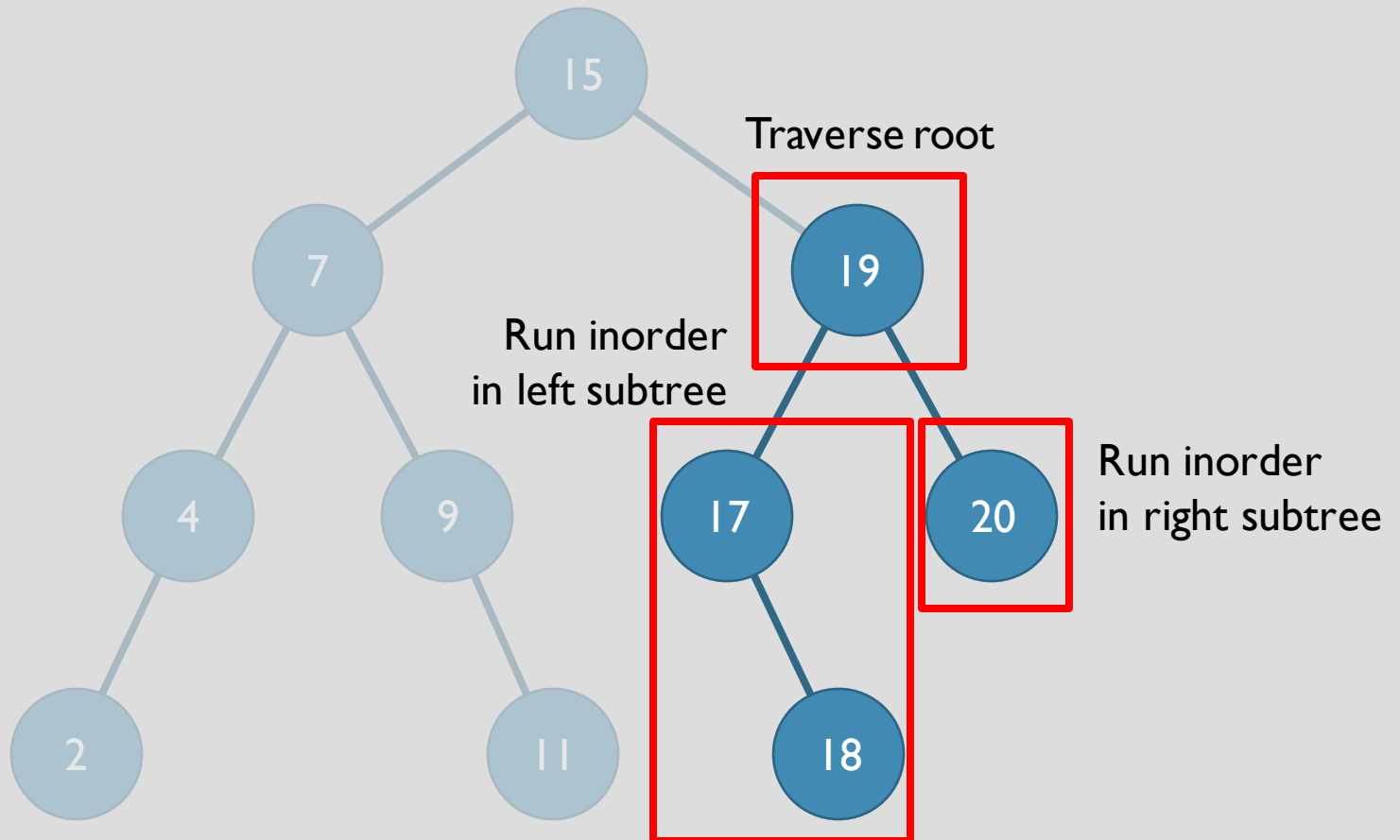


Run inorder
in left subtree

Run inorder
in right subtree

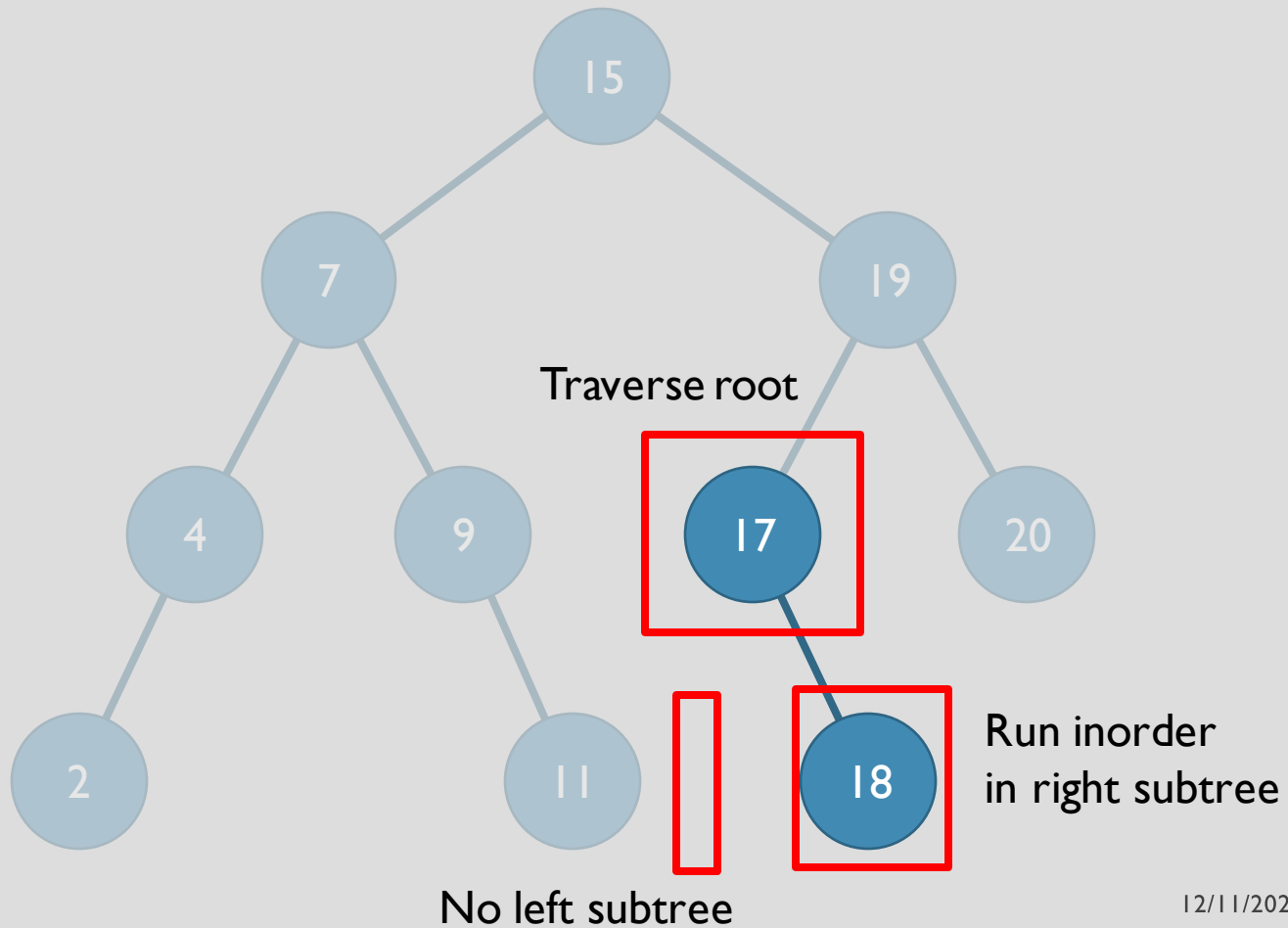
INORDER TRAVERSAL

2, 4, 7, 9, 11, 15,



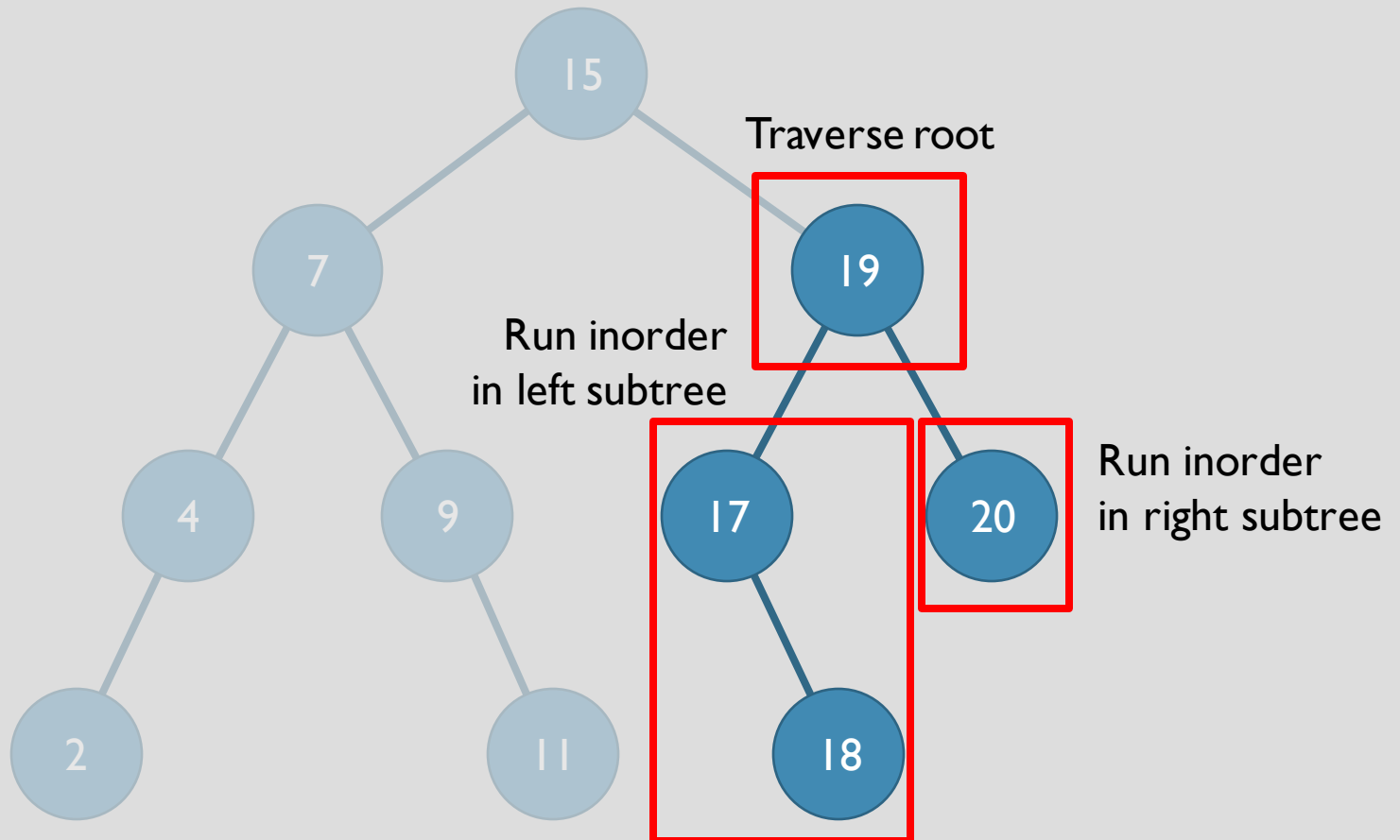
INORDER TRAVERSAL

2, 4, 7, 9, 11, 15, 17, 18,



INORDER TRAVERSAL

2, 4, 7, 9, 11, 15, 17, 18, 19, 20 ← Sorted!!!

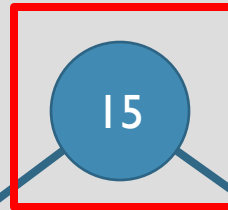


INORDER IN BST

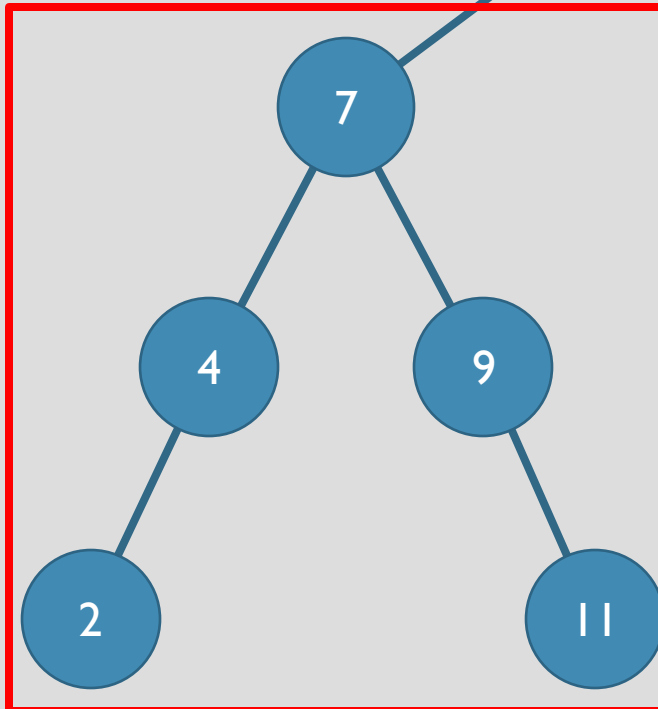
- An inorder traversal in a BST gives us a sorted sequence of the given data
- Thus, an inorder traversal can be used as a correctness test of the BST you created
- Remember, sorted sequence appears for **inorder traversal in BSTs only**

POSTORDER TRAVERSAL: LEFT, RIGHT, ROOT

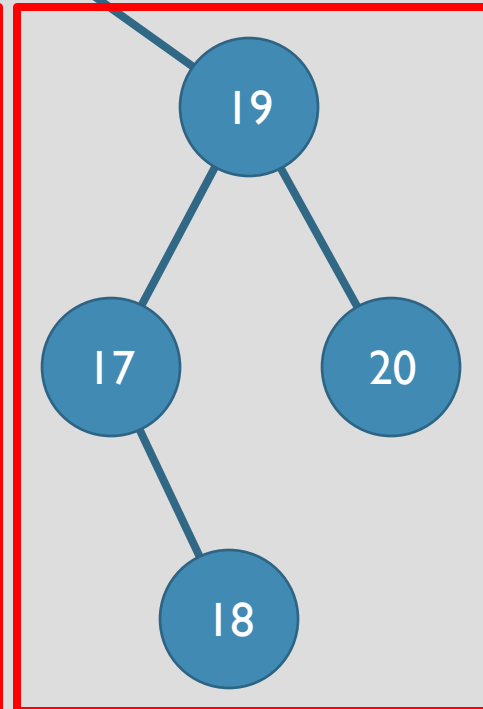
Traverse root



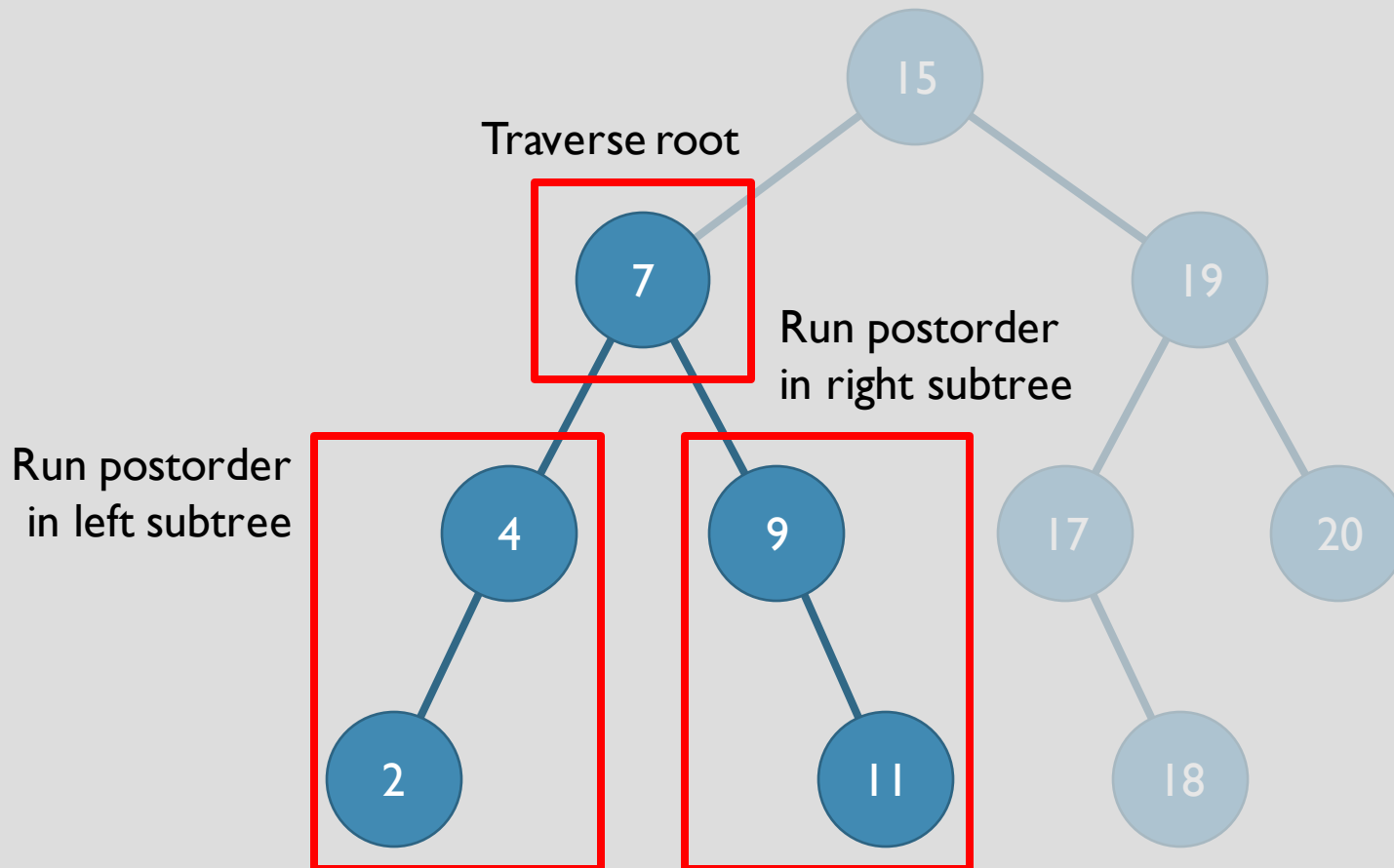
Run postorder
in left subtree



Run postorder
in right subtree

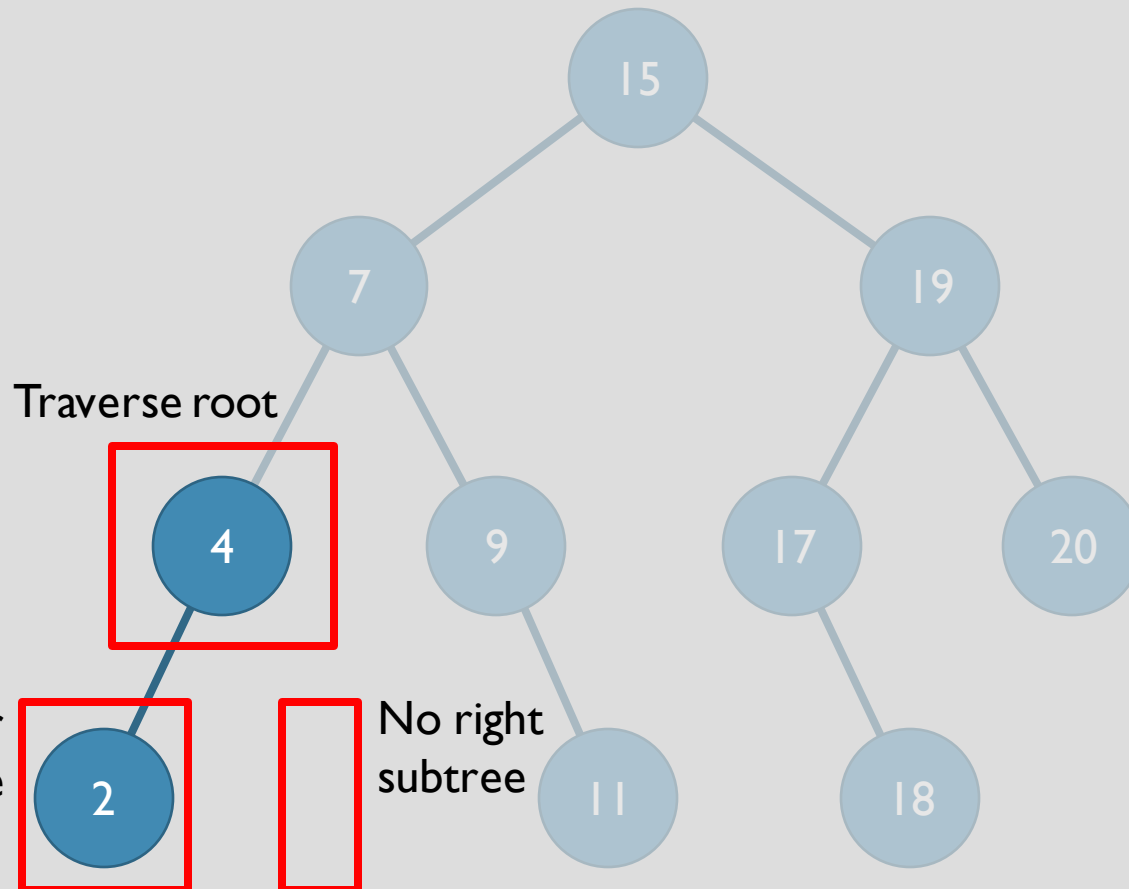


POSTORDER TRAVERSAL



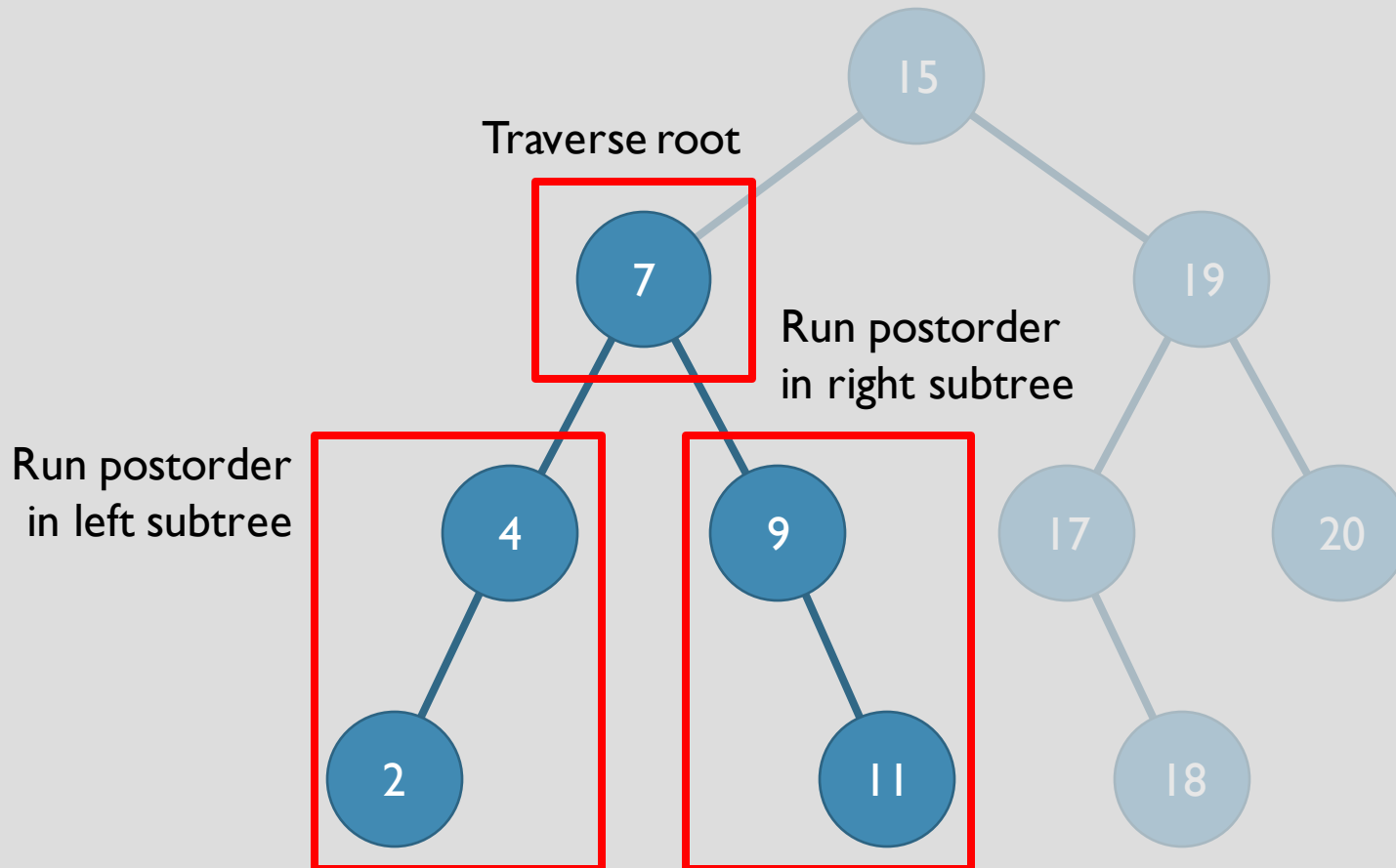
POST ORDER TRAVERSAL

2, 4,



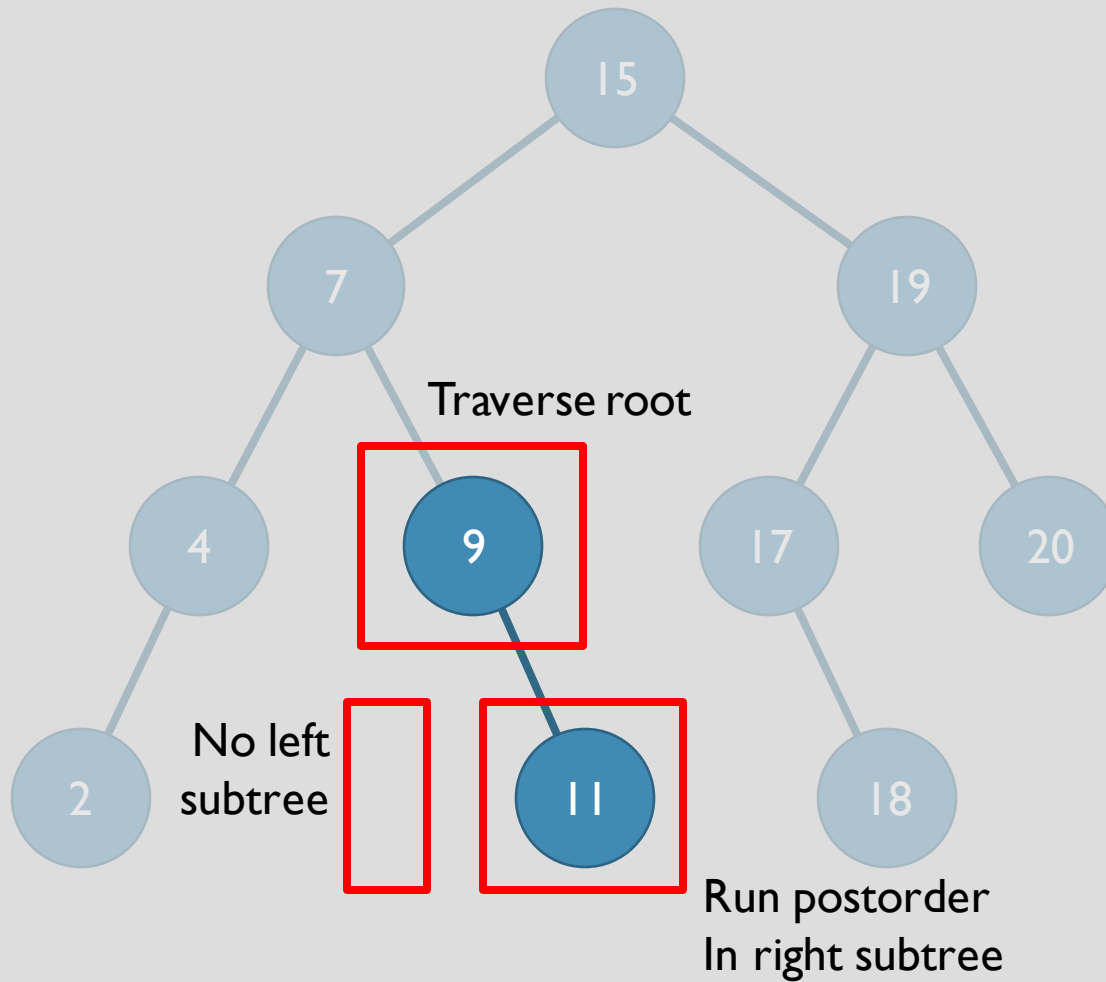
POSTORDER TRAVERSAL

2, 4,



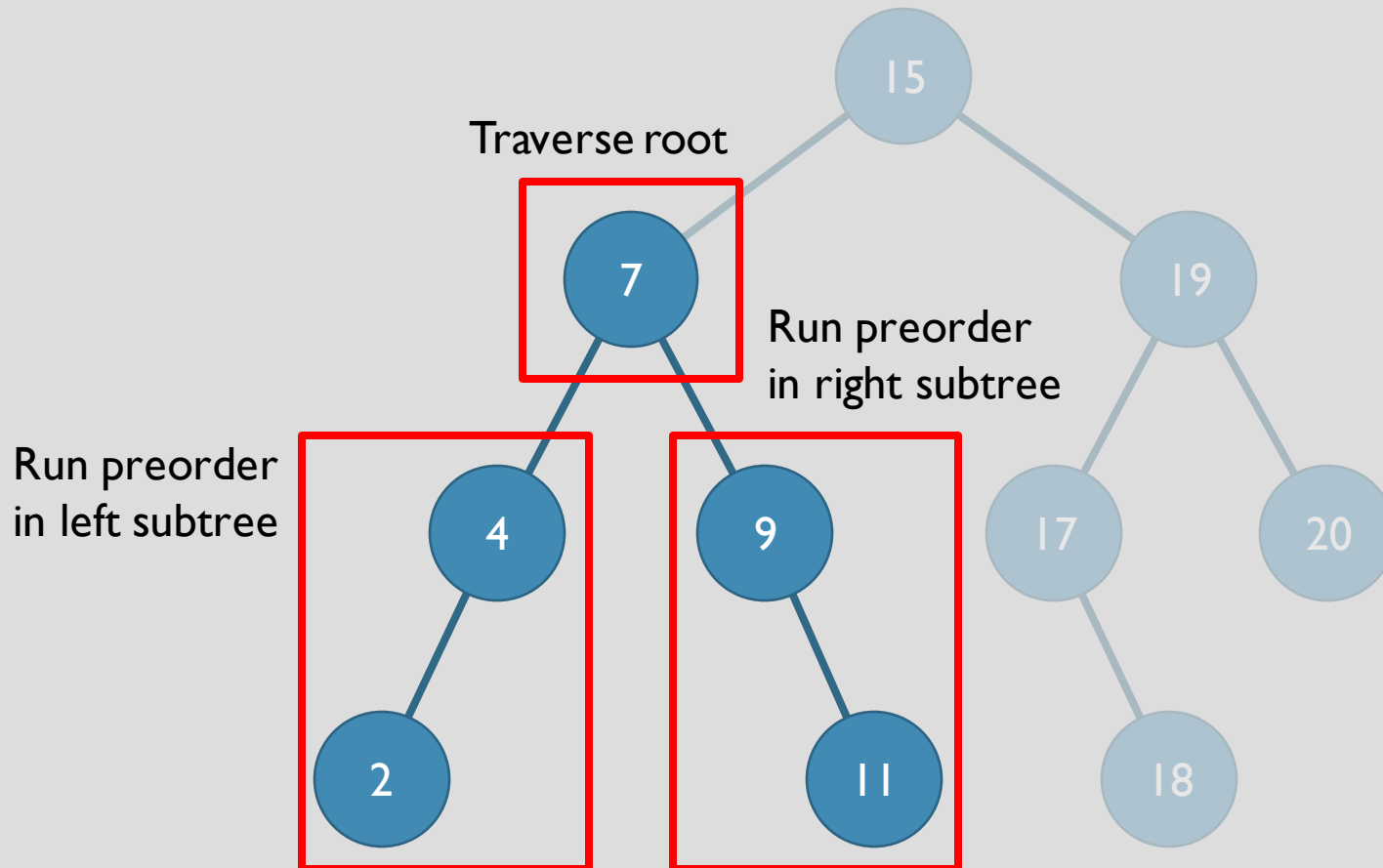
POSTORDER TRAVERSAL

2, 4, 11, 9,



POSTORDER TRAVERSAL

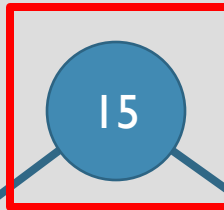
2, 4, 11, 9, 7,



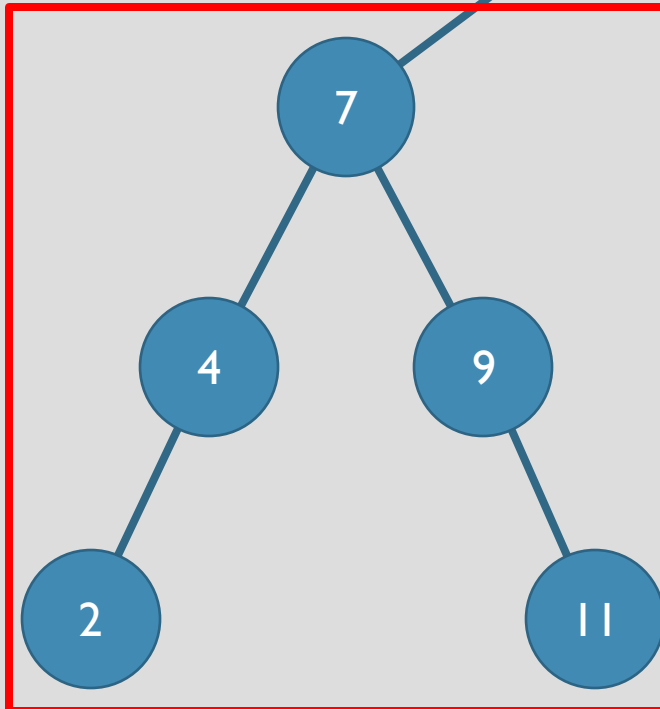
POSTORDER TRAVERSAL

2, 4, 11, 9, 7,

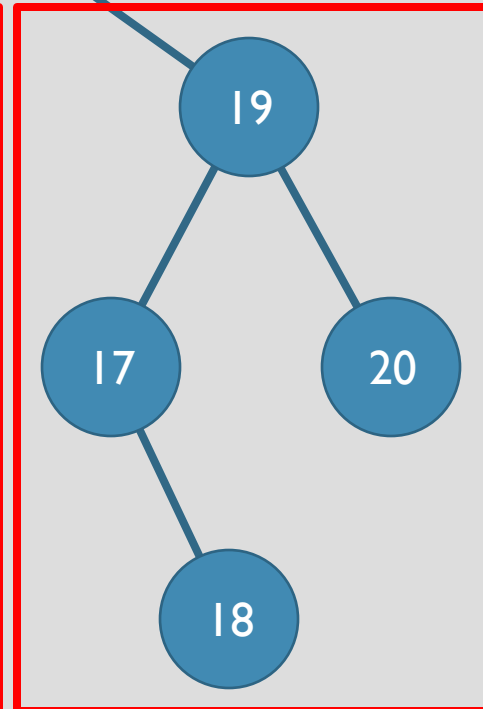
Traverse root



Run postorder
in left subtree

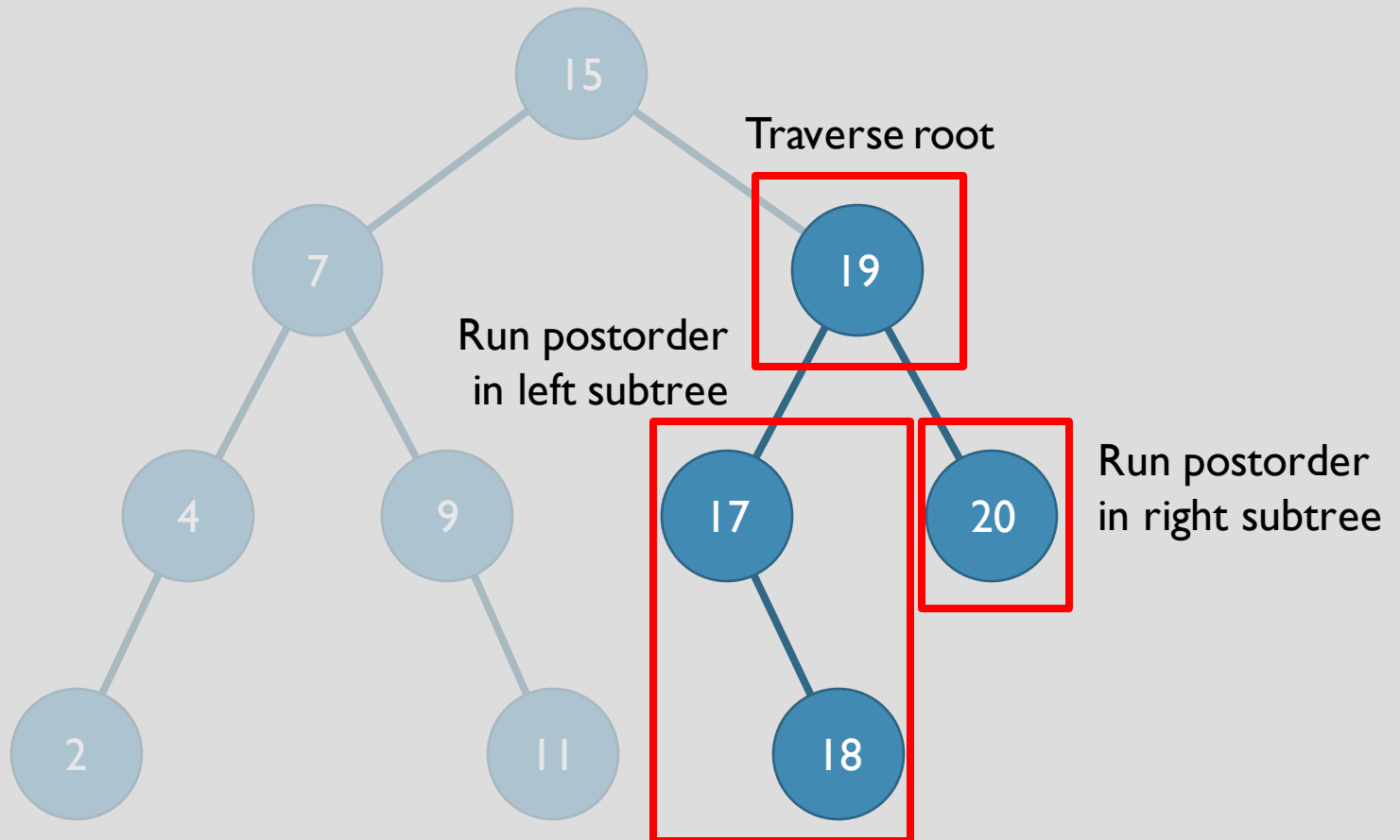


Run postorder
in right subtree



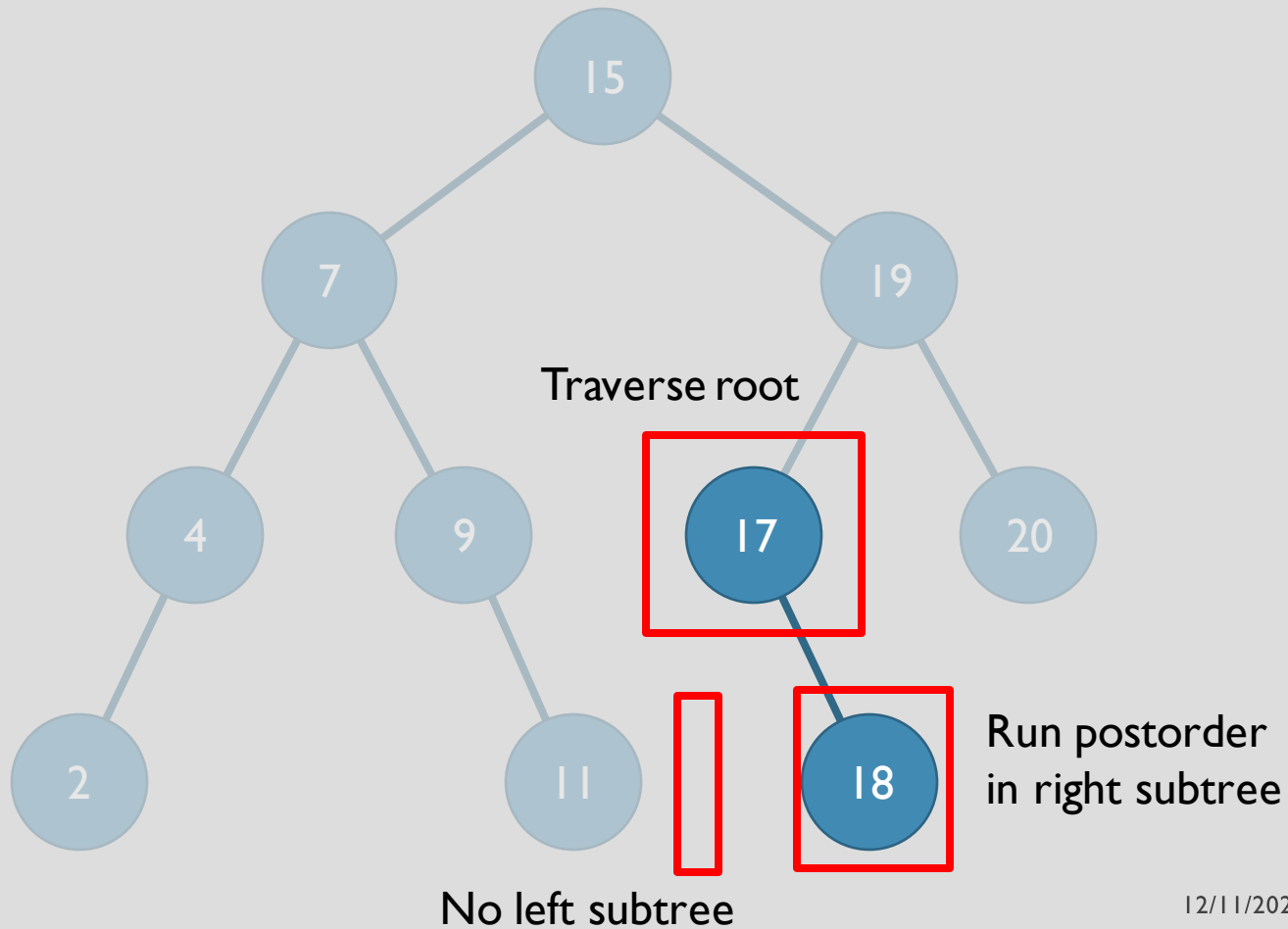
POSTORDER TRAVERSAL

2, 4, 11, 9, 7,



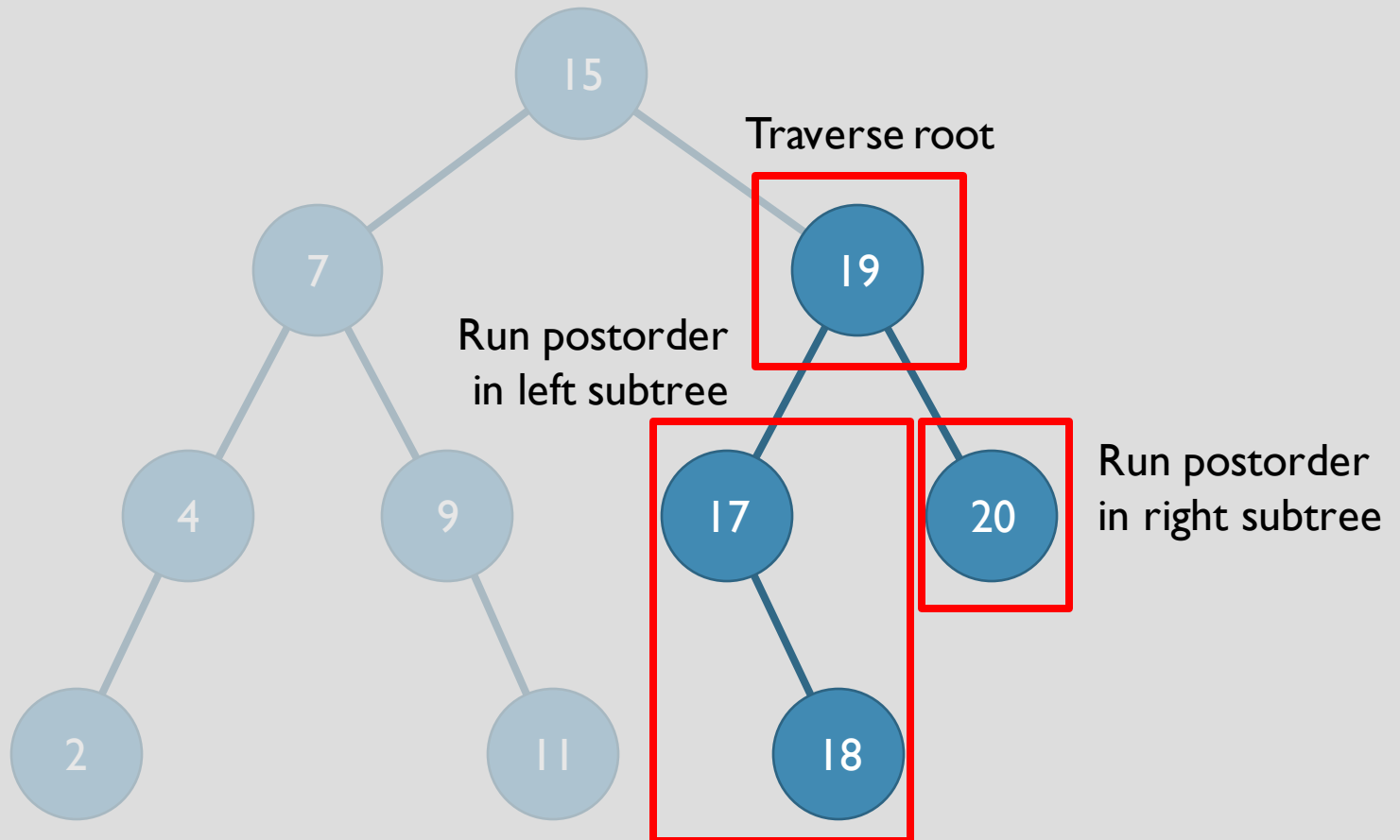
POSTORDER TRAVERSAL

2, 4, 11, 9, 7, 18, 17,



POSTORDER TRAVERSAL

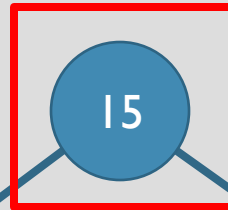
2, 4, 11, 9, 7, 18, 17, 20, 19,



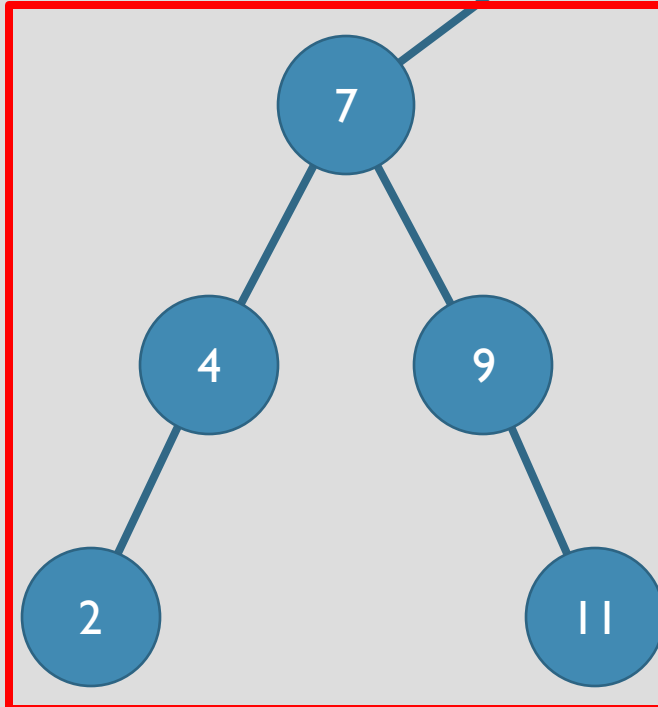
POSTORDER TRAVERSAL

2, 4, 11, 9, 7, 18, 17, 20, 19, 15

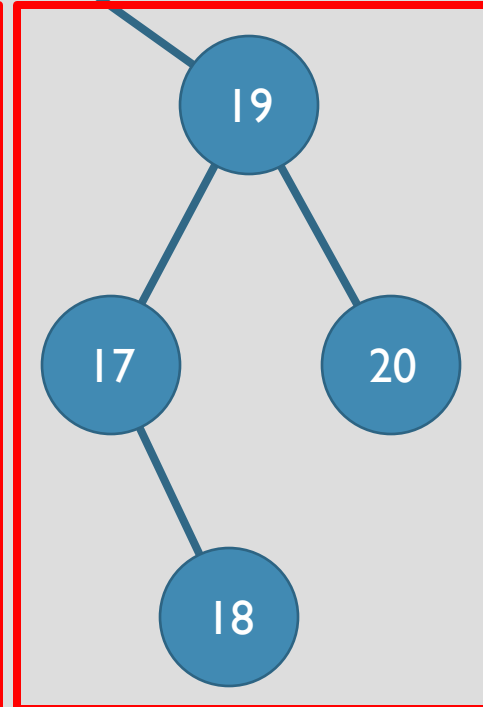
Traverse root



Run postorder
in left subtree



Run postorder
in right subtree



NOTES ABOUT TREE TRAVERSAL

- Tree traversal algorithms are also applicable to any ordered trees, even m -ary trees where $m \geq 3$
- How to achieve inorder in this case?
 - Use the order “First child – Root – Other children”